

ExpertWeave

总体目标

本项目面向 Mixture-of-Experts 模型中的 Expert Offloading 场景。

MoE 模型通过稀疏激活机制，使每个 token 只需要调用少量专家进行计算，从而在显著扩大模型参数规模的同时控制实际计算量。然而，稀疏计算并不意味着所有专家参数都可以忽略。随着专家数量和专家规模不断增加，完整模型的权重通常无法同时常驻于有限的 GPU 显存中。因此，推理系统需要将部分专家权重存储在 CPU 内存或其他层级存储介质中，并根据模型运行时产生的路由结果，将当前需要使用的专家动态加载到 GPU。

在这一过程中，专家权重需要通过 PCIe 等互连链路从 CPU 内存传输到 GPU 显存。由于单个专家通常包含多个规模较大的投影矩阵，并且不同层、不同 token 和不同请求可能持续激活不同专家，专家权重的动态 Onload 可能产生显著的数据搬运开销。当某个专家尚未加载完成时，依赖该专家的计算必须暂停等待，从而产生专家加载阻塞。这种阻塞会降低 GPU 的有效利用率，增加请求延迟，并限制系统在有限显存条件下运行大规模 MoE 模型的能力。

本项目的**核心目标**是设计一种适用于 MoE 专家权重的实时无损压缩、传输和恢复机制。该机制以压缩形式保存需要被 Offload 的专家权重，当某个专家被模型路由器选中时，系统只需要将压缩后的权重数据传输到 GPU，并在 GPU 上或靠近 GPU 的位置完成解压缩和权重恢复。整个过程必须保证专家权重能够被完整恢复，不能改变原始浮点数的数值内容或比特表示，也不能影响模型推理结果的正确性。

本项目首先希望通过压缩减少专家权重在 PCIe 链路上的实际传输数据量。专家权重通常以 BF16、FP16 或其他浮点格式存储，其符号位、指数位和尾数位可能具有不同的统计分布。同时，同一个专家内部的不同神经元、同一层中的不同专家以及不同投影矩阵之间，也可能存在可以利用的结构性或统计相关性。项目将研究这些特征能否被稳定地转化为无损压缩收益，从而减少每次专家 Onload 所需要传输的字节数。

但是，减少传输数据量并不是本项目的最终目标。压缩机制本身会引入额外开销，包括压缩数据定位、元数据读取、解压缩 kernel 启动、临时内存分配、权重布局恢复、额外数据复制和设备同步等。如果这些额外开销大于压缩所节省的 PCIe 传输时间，那么即使压缩率较高，系统仍然不会获得实际性能提升。因此，本项目将压缩率视为一种实现手段，而不是独立的优化目标。

本项目将 Expert-Ready Latency 作为核心组件级性能指标。Expert-Ready Latency 是指从系统确定需要加载某个专家开始，到该专家的完整权重已经位于 GPU 显存中，并能够

被后续计算 **kernel** 直接使用为止所经历的总时间。该指标不仅包含 **PCIe** 数据传输时间，也包含专家查找、元数据处理、任务提交、解压恢复、数据布局转换和同步等待等完整路径上的开销。与单独测量传输带宽或解压吞吐量相比，**Expert-Ready Latency** 更能够反映压缩方案是否真正改善了专家动态加载过程。

项目还将研究如何将专家传输和解压缩过程融入 **MoE** 推理流水线。理想情况下，系统不应等待整个压缩专家传输完成后再开始解压，而应将专家数据划分为多个块，使数据传输和解压缩能够以流式方式执行。部分压缩数据到达 **GPU** 后即可立即开始恢复，而剩余数据继续通过 **PCIe** 传输。与此同时，专家加载过程还应尽可能与 **Attention** 计算、其他层计算或其他专家计算重叠，从而减少真正暴露在请求关键路径上的等待时间。

本项目最终需要在真实 **MoE Offloading** 系统中验证压缩机制是否能够带来端到端推理收益。评估不仅需要关注单个专家的压缩率和加载时间，也需要测量完整请求的首 **token** 延迟、每 **token** 延迟、平均延迟、尾延迟、吞吐量、**GPU** 利用率和专家加载阻塞时间。只有当压缩机制降低了真实工作负载中的请求关键路径，或者在相同服务延迟约束下提高了系统吞吐量，才能认为该方案实现了最终目标。

因此，本项目的最终判据是判断：专家权重经过无损压缩后，压缩数据的传输、解压恢复以及相关运行时管理所消耗的总时间，能否低于直接传输原始专家权重的时间，并进一步转化为真实 **MoE** 推理系统中的端到端性能提升。

研究问题

本项目将围绕 **MoE Expert Offloading** 中的专家权重压缩、动态传输和运行时执行机制展开，主要研究以下问题。

（1）**MoE** 专家权重的无损可压缩性问题

现有 **MoE** 模型中的不同专家通常具有相同的网络结构，但会在训练过程中学习不同的参数分布和功能。由于专家之间具有相同的矩阵形状和浮点表示形式，不同专家之间可能存在统计相似性；与此同时，单个专家内部也可能存在指数分布集中、尾数模式重复或局部数值相关等特征。这些特征为无损压缩提供了潜在空间。

本项目需要首先分析 **MoE** 专家权重是否具有稳定且可复现的无损可压缩性。需要区分哪些压缩收益来自通用浮点数分布，哪些收益真正来自 **MoE** 专家的结构特点。例如，如果不同专家之间的相似性无法带来额外收益，而对单个专家独立使用通用熵编码已经能够达到近似最优结果，那么设计复杂的跨专家预测方法就没有必要。

研究还需要覆盖不同模型、不同层、不同专家和不同投影矩阵。某种方法可能只在个别层或个别矩阵上有效，却无法推广到完整模型。因此，需要通过发现集和独立测试集验证压缩规律是否具有泛化能力，避免根据少量样本得到过度乐观的结论。

这一问题需要回答的本质是，MoE 专家权重中是否存在足够强、足够稳定并且能够被高效利用的无损压缩信号。如果专家权重本身的可压缩空间有限，那么后续系统优化将受到根本限制。

难度：容易到中等。

（2）面向实时专家加载的压缩格式设计问题

传统压缩算法通常主要关注压缩率、存储容量或离线压缩速度，但 MoE Expert Offloading 对压缩格式提出了不同要求。专家权重位于模型执行关键路径上，当路由结果确定后，系统需要尽快完成专家加载。压缩格式即使能够获得很高的压缩率，如果解压延迟过长、元数据过大或无法高效地在 GPU 上并行恢复，也不适合实时 Offloading。

本项目需要设计一种面向低延迟加载的无损压缩格式。该格式应具有较低的元数据开销，能够支持连续内存传输，并能够通过 GPU kernel 高度并行地完成解码。压缩数据还应尽可能直接恢复到专家计算所使用的最终内存布局中，避免先解压到临时缓冲区，再进行额外的数据复制或格式转换。

压缩粒度也是一个关键问题。以完整专家为单位进行压缩，可能具有更好的全局统计特征，但随机访问和流式解压能力较差。以固定大小数据块为单位进行压缩，可以支持传输和解压流水化，但会引入更多码表和元数据开销。项目需要在压缩率、随机访问能力、解压并行度和元数据成本之间进行权衡。

此外，压缩格式必须正确处理所有可能出现的浮点比特模式，包括特殊值、非规范数以及可能存在的 NaN payload。项目所要求的无损恢复不仅是数值上近似一致，而是需要在定义的无损边界内完整恢复原始权重表示。

这一问题需要回答的是，如何设计一种以最小 Expert-Ready Latency 为目标，而不是单纯以最高压缩率为目标的专家权重压缩格式。

难度：中等。

（3）PCIe 传输与解压缩的协同优化问题

采用压缩传输后，系统实际上是在传输开销和计算开销之间进行转换。原始方案需要传输更大的权重数据，但几乎不需要额外解码；压缩方案减少了 PCIe 数据量，却增加了解压

缩和运行时管理工作。只有当减少的传输时间大于新增的解压缩、元数据处理、额外复制和同步开销时，压缩方案才具有局部性能收益。

本项目需要分析专家加载过程中各个阶段的执行关系，并研究如何减少串行开销。最直接的实现方式是先将整个压缩专家传输到 GPU，再启动解压 kernel，但这种方式会使传输和解压完全串行，难以充分利用压缩带来的带宽收益。更合理的方式是将专家划分为多个压缩块，在每个数据块到达 GPU 后立即启动对应的解压任务，使 PCIe 传输和 GPU 解压形成流水线。

项目还需要考虑解压缩由 CPU 还是 GPU 执行。如果在 CPU 侧解压，传输到 GPU 的仍然是原始权重，无法减少 PCIe 数据量；如果在 GPU 侧解压，则会占用 GPU 计算资源和显存带宽。不同硬件平台的 PCIe 带宽、CPU 性能、GPU 算力和显存带宽不同，最优方案也可能不同。

此外，解压 kernel 的启动次数、数据块大小、CUDA Stream 数量和同步方式都会影响最终性能。过小的数据块可以更早开始解压，但会产生大量 kernel launch 和调度开销；过大的数据块可以提高解压吞吐量，却降低传输与解压之间的重叠程度。因此，需要通过分析和实验确定适合不同专家规模与硬件环境的流水线配置。

这一问题需要回答的是，如何将传统的串行专家加载过程转化为分块传输、流式解压和前后台计算重叠的执行流水线。

难度：困难。

（4）Expert-Ready Latency 的建模与关键路径分析问题

仅仅报告压缩率、PCIe 带宽或解压吞吐量，无法准确判断专家是否能够更快参与计算。真实专家加载路径中还存在地址查找、元数据解析、任务提交、内存分配、kernel 启动和同步等待等开销。这些开销中的一部分可能很小，但也可能因为引入了全局同步而成为关键路径。

本项目需要建立完整的 Expert-Ready Latency 测量和分析方法。首先需要明确测量的起点和终点，确保不同方案之间的时间边界一致。其次需要区分原始数据传输时间、压缩数据传输时间、解压时间以及专家真正可计算的时间，避免只测量其中最有利的部分。

研究还需要分析哪些开销能够被其他计算隐藏，哪些开销必然暴露在关键路径上。例如，某个压缩方案虽然解压时间较长，但如果解压可以完全与前一层计算重叠，那么它对请求延迟的影响可能很小。相反，一个看似很短的同步操作，如果阻塞了整个计算流，也可能显著增加专家准备时间。

单专家微基准也不能完全代表真实系统行为。当多个专家同时需要加载时，它们会竞争 PCIe 带宽、GPU 解压资源和显存带宽。系统可能出现排队、阻塞和资源争用，导致单专家实验中观察到的收益无法保持。因此，需要进一步测量多专家并发加载和连续层级加载条件下的 Expert-Ready Latency。

这一问题需要回答的是，压缩方案是否真正缩短了专家从被请求到可以执行计算的完整时间，而不是只优化了某个孤立阶段。

难度：中等到困难。

（5）专家访问模式与自适应加载策略问题

不同专家在真实推理工作负载中的访问频率并不相同。部分专家可能被大量 token 高频激活，具有较高的复用概率；另一些专家则可能长期不被访问，仅在少量特殊输入中出现。热门专家更适合长期驻留在 GPU，而冷门专家更可能成为 Offloading 和压缩传输的对象。

不同专家的压缩效果也可能不同。某些专家具有较高压缩率和较低解压成本，适合采用压缩传输；某些专家压缩率有限，使用压缩后无法覆盖解压开销，直接传输原始权重反而更快。因此，固定地对所有专家采用同一种压缩策略可能不是最优选择。

本项目需要研究如何根据专家热度、专家大小、压缩率、复用距离、当前 GPU 负载和 PCIe 竞争情况，动态选择专家的存储和加载方式。系统可以在常驻 GPU、原始权重传输、压缩权重传输和提前预取等策略之间进行选择。当 GPU 计算资源空闲而 PCIe 拥塞时，压缩传输可能更有价值；当 GPU 已经高度繁忙而 PCIe 带宽充足时，额外解压可能造成更大干扰。

项目还需要研究是否可以通过离线 profiling 和在线统计结合的方式预测每个专家的加载成本。离线阶段可以记录专家压缩率、解压吞吐量和权重大小，在线阶段则根据实际专家访问频率、队列状态和硬件利用率调整策略。

这一问题需要回答的是，如何根据专家自身特征和系统运行状态，自适应地判断一次专家加载是否应当采用压缩传输。

难度：困难。

（6）专家压缩、缓存和 Offloading 调度的联合优化问题

专家压缩会改变专家缓存系统的容量和加载成本，因此不能与缓存和调度策略完全分离。压缩后的专家权重在 CPU 内存或其他存储层级中占用更少空间，使系统能够保留更多专家副本，但专家被加载到 GPU 并参与计算前，仍然需要恢复为完整权重。

本项目需要研究压缩副本和解压副本之间的管理方式。系统可以只在 CPU 中保存压缩副本，也可以在 GPU 上短暂保存压缩数据，还可能同时维护压缩副本和已解压副本。不同方式会影响内存占用、加载延迟和替换开销。如果专家被驱逐出 GPU 后还需要重新压缩，那么压缩开销也可能进入运行时关键路径。

专家缓存替换策略也需要考虑加载成本。传统缓存策略通常主要依据最近访问时间或访问频率决定驱逐对象，但在压缩 Offloading 场景中，不同专家的重新加载成本可能不同。一个体积较大、压缩率低且解压较慢的专家，即使访问频率不高，也可能值得继续保留；一个能够快速压缩加载的专家，则更适合作为驱逐对象。

预取策略同样会受到压缩机制影响。压缩数据更小，可能使系统能够在相同时间窗口内预取更多专家；但如果预取错误，解压过程仍然会消耗 GPU 资源。因此，预取决策需要同时考虑路由预测准确率、压缩传输成本、解压成本和对前台计算的干扰。

这一问题需要回答的是，如何联合考虑专家压缩率、访问热度、缓存容量和加载代价，优化专家的驻留、驱逐、预取和传输过程。

难度：困难。

（7）解压缩对 GPU 资源的干扰问题

GPU 侧解压缩虽然能够减少 PCIe 传输数据量，但会占用 GPU 的计算和内存资源。解压 kernel 可能消耗流处理器、显存带宽、L2 Cache、CUDA Stream 调度资源和临时工作空间。当 GPU 同时执行 Attention、GEMM、All-to-All 通信或其他专家计算时，解压任务可能与前台推理任务发生竞争。

这种资源竞争可能使压缩方案产生隐藏成本。例如，解压任务即使没有直接增加 Expert-Ready Latency，也可能降低正在执行的 GEMM 性能，使模型其他部分变慢。最终表现为专家加载变快，但完整请求反而没有加速。压缩系统必须避免将 PCIe 瓶颈简单转移为 GPU 计算瓶颈或显存带宽瓶颈。

本项目需要通过并发微基准和真实推理测试，分析解压 kernel 对不同前台算子的影响。需要研究不同 CUDA Stream 优先级、解压并发度、数据块大小和执行时机，判断是否可以限制解压任务对前台计算的干扰。当 GPU 负载较高时，系统可能需要降低解压并发度，延迟部分预取任务，或者直接选择原始权重传输。

解压过程所需的临时工作空间也需要纳入评估。MoE Offloading 本身通常发生在显存紧张的场景，如果压缩方案需要大量额外缓冲区，可能进一步挤占专家缓存、激活值或 KV Cache 空间，从而抵消压缩带来的容量优势。

这一问题需要回答的是，压缩减少了 PCIe 数据传输之后，是否会将系统瓶颈转移到 GPU 计算、显存带宽或显存容量上。

难度：困难。

（8）真实 MoE 工作负载下的端到端收益问题

单个专家的传输和解压微基准只能验证局部机制是否有效，不能直接证明完整 MoE 推理系统能够获得加速。真实系统还受到路由分布、批处理大小、每个 token 激活的专家数量、Prefill 和 Decode 阶段差异、专家并行方式、All-to-All 通信、专家预取准确率和请求到达模式等多种因素影响。

在 Prefill 阶段，大量 token 可能同时激活多个专家，系统更容易出现批量专家加载和带宽竞争。在 Decode 阶段，每个迭代处理的 token 数较少，但专家加载可能更加频繁，并且单次阻塞更容易直接影响每 token 延迟。因此，同一种压缩机制在 Prefill 和 Decode 中可能表现出完全不同的收益。

本项目需要在不同模型规模、专家数量、专家大小、批处理规模和请求负载下进行系统评估。需要覆盖专家访问高度集中和高度分散的场景，也需要覆盖预取准确和预取失败的情况。除了平均性能，还应关注尾延迟，因为解压排队或资源争用可能使少量请求产生严重延迟。

项目还需要判断压缩机制是否能够支持原本由于显存或带宽限制而无法运行的配置。例如，在相同 GPU 显存下缓存更多专家，或者在相同延迟约束下支持更大的并发批次，也可以视为有价值的系统收益。不过，这类容量收益应与直接的延迟和吞吐收益分别报告，避免将不同类型的改进混为一谈。

这一问题需要回答的是，在真实 MoE Expert Offloading 系统中，专家压缩是否能够稳定降低请求关键路径，并最终改善推理延迟、吞吐量或系统容量，而不是只在孤立微基准中取得局部优势。

难度：中等到困难。

总体研究主线

本项目的研究可以划分为三个递进层次。第一层是数据层面的可压缩性研究，主要判断 MoE 专家权重中是否存在稳定、可泛化并且能够被高效利用的无损压缩空间。第二层是组件层面的实时加载研究，主要解决如何使压缩数据传输、解压恢复和权重准备的总时间低于原始权重直接传输时间。第三层是系统层面的推理优化研究，主要验证 Expert-Ready Latency 的降低是否能够真正转化为请求延迟下降、吞吐量提升或系统容量增加。

综合而言，本项目旨在设计并实现一种面向 MoE Expert Offloading 的实时无损权重压缩机制。该机制通过减少 PCIe 传输数据量、优化 GPU 解压执行、控制运行时资源干扰，并与专家缓存和 Offloading 调度协同工作，从而降低专家加载阻塞和 Expert-Ready Latency，最终在真实 MoE 推理工作负载中获得可验证的端到端性能收益。